

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data

storage and retrieval efficiency. (BL3, BL4)

Activity Tool : Comparative Analysis (Low)

Assessment Tool Weightage: 10%

Code of Conduct:

I M.Keerthi reddy (192525243) certify that this submission is my

original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M.Keerthi reddy

Name :M.keerthi reddy Reg. No: 192525243

QUESTIONS

Q.No

Question

Bloom's Level

1

Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.

4 – Analyze

2

Differentiate between clustered indexing and non-clustered indexing with suitable database examples.

4 – Analyze

3

Compare primary indexing and secondary indexing based on storage requirements and search performance.

4 – Analyze

4

Analyze the differences between static hashing and dynamic hashing in database systems.

4 – Analyze

5

Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.

4 – Analyze

6

Differentiate linear hashing and extendible hashing in terms of bucket

management and scalability.

4 – Analyze

7

Compare dense indexing and sparse indexing with respect to memory usage and access speed.

4 – Analyze

8

Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.

4 – Analyze

9

Compare single-level indexing and multi-level indexing for efficient database retrieval operations.

4 – Analyze

10

Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.

5 – Evaluate

RUBRICS FOR CALCULATION:

Criteria

Weightage

(%)

Level 4

(Exemplary)

Level 3

(Proficient)

Level 2

(Developing)

Level 1

(Beginning)

Scope of Items

20%

Selects highly

relevant items

with clear scope

and justification

Selects

appropriate items

with minor gaps

Selection is

limited or

partially relevant

Items are

unclear or not

relevant

Comparison

Depth

25%

Provides detailed

and

comprehensive

comparison across

multiple aspects

Provides adequate

comparison with

some depth

Limited

comparison with

few aspects

Very minimal or

no comparison

Use of Evidence

20%

Supports

comparison with

accurate data,

examples, or

references

Uses some

supporting

evidence with

minor gaps

Limited or weak

supporting

evidence

No supporting

evidence

provided

Critical Thinking

20%

Demonstrates

strong critical

thinking with clear

Shows reasonable

interpretation

with minor gaps

Basic

interpretation;

lacks depth

No meaningful

interpretation

Criteria

Weightage

(%)

Level 4

(Exemplary)

Level 3

(Proficient)

Level 2

(Developing)

Level 1

(Beginning)

interpretation and

reasoning

Clarity of

Presentation

15%

Well-organized,

clear, and logically

structured

presentation

Generally clear

with minor issues

Somewhat

unclear or poorly

structured

Disorganized and
difficult to
understand

Q1. Sequential File Organization vs Heap (Pile) File Organization

Bloom's Level: 4 – Analyze

Comparison Table

Aspect

Sequential File Organization

Heap File Organization

Definition

Records stored physically in order of a
key field

Records stored in the order they arrive,
no fixed order

Insertion

Slow — may require shifting/rewriting to
keep order, or use of overflow area

Very fast — new record simply appended
at the end

Deletion

Requires reorganization or
tombstone/marker to keep file sorted

Fast — mark as deleted; space can be
reused later

Retrieval (search)

Efficient for key-based search using
binary search — $O(\log n)$

Inefficient — full linear scan needed —

$O(n)$

Range queries

Very efficient since data is already sorted

Poor — every record must be scanned

Storage overhead

Low, but periodic reorganization needed

Minimal overhead but wasted space

from deleted records

Best use case

Batch processing, reporting, payroll

systems

Logging, staging/temporary data, bulk

loading before indexing

Explanation

Sequential file organization stores records physically sorted by a key attribute (e.g., roll number), which allows

fast key-based and range retrieval using binary search, but insertions and deletions are costly because

maintaining sort order may require shifting records or using an overflow file that is periodically merged back.

Heap (pile) file organization simply appends new records wherever free space is available, with no ordering. This

makes insertion an $O(1)$ operation, but every retrieval — even for a single record — typically requires scanning

the entire file ($O(n)$), which becomes very expensive as the file grows.

Example(s)

-

A student database sorted by roll number (sequential) supports fast rank-list generation, while a temporary log file that only needs to record events as they occur (heap) benefits from fast appends.

Justification / Critical Analysis

The choice between the two is a direct trade-off between write performance and read performance. Sequential

organization is preferable when the workload is read-heavy and queries are key-based or range-based, whereas

heap organization is preferable when the workload is write-heavy and records are rarely searched individually.

In practice, heap files are often used as a staging area before bulk-loading into an indexed structure that

combines fast writes with fast reads.

Q2. Clustered Indexing vs Non-Clustered Indexing

Bloom's Level: 4 – Analyze

Comparison Table

Aspect

Clustered Indexing

Non-Clustered Indexing

Physical order

Data rows are physically sorted and
stored in the same order as the index

Index is a separate structure; data rows
are not physically reordered

Number per table

Only one clustered index per table (since
data can have only one physical order)

Multiple non-clustered indexes can exist
on the same table

Storage

No separate data copy — index IS the
data arrangement

Extra storage needed for index structure
with pointers to actual rows

Retrieval speed

Faster for range queries because related
rows are physically adjacent

Slightly slower — requires an extra
lookup (pointer to row) after finding the
index entry

Example (SQL Server)

Primary Key by default becomes a

clustered index

CREATE INDEX on a non-key column, e.g.,

dept or email

Explanation

A clustered index determines the physical storage order of table rows — the leaf level of the index IS the data

itself. Because of this, a table can have only one clustered index (commonly built on the primary key).

A non-clustered index is a separate structure containing the indexed column values along with a pointer (row

locator) to where the actual row is physically stored. This allows several non-clustered indexes to coexist on the

same table.

Example(s)

-

In a Students table, roll_no (primary key) is typically the clustered index, so records are physically stored

roll-number order — ideal for range reports like 'roll numbers 100 to 200'.

-

A non-clustered index on the dept column allows fast lookup of 'all students in CSE' without reordering

the whole table.

Justification / Critical Analysis

Below is real evidence generated using SQLite (a relational engine) on a 2,000-row Students table, showing how

the query optimizer chooses different access paths depending on which column is indexed.

Code & Output Evidence (SQLite demonstration)

The following script creates a Students table (roll_no INTEGER PRIMARY KEY, name, dept, marks) with 2,000 rows and

inspects the query execution plan before and after indexing dept.

```
CREATE TABLE students (roll_no INTEGER PRIMARY KEY, name TEXT, dept TEXT, marks INTEGER);
```

```
-- 2000 rows inserted
```

```
EXPLAIN QUERY PLAN SELECT * FROM students WHERE dept='CSE';
```

```
>> SCAN students          -- (before creating index: full scan)
```

```
CREATE INDEX idx_dept ON students(dept);
```

```
EXPLAIN QUERY PLAN SELECT * FROM students WHERE dept='CSE';
```

```
>> SEARCH students USING INDEX idx_dept (dept=?) -- non-clustered index used
```

```
EXPLAIN QUERY PLAN SELECT * FROM students WHERE roll_no=500;
```

```
>> SEARCH students USING INTEGER PRIMARY KEY (rowid=?) -- clustered access via PK
```

Full table scan time (no index) : 0.195 ms

Indexed lookup time : 0.056 ms (~3.5x faster)

Output interpretation: Without an index, SQLite's query planner performs a full SCAN of every row. After

CREATE INDEX idx_dept, the same query switches to a SEARCH using the index — a non-clustered, secondary

structure. Lookup by roll_no (the PRIMARY KEY, which is clustered in SQLite's rowid table) is already an indexed

SEARCH by default. The measured timing confirms indexed access is faster than a full scan even on a modest

dataset, and the gap widens sharply as table size grows.

Q3. Primary Indexing vs Secondary Indexing

Bloom's Level: 4 – Analyze

Comparison Table

Aspect

Primary Indexing

Secondary Indexing

Built on

Ordering (primary/key) attribute of a sequentially organized file

Any non-ordering attribute, which may or may not be a candidate key

Type

Usually sparse — one index entry per data block

Usually dense — one index entry per record

Storage requirement

Lower, since fewer entries are stored

(one per block)

Higher, since an entry is stored for every record

Search performance

Fast — index is small enough to often fit in memory; binary search on index then scan the block

Slower per index lookup due to larger index size, but enables search on non-key attributes

Data file dependency

Requires the data file to be physically sorted on the indexed field

Data file need not be sorted on the indexed field

Explanation

A primary index is built on the ordering key of a file that is itself physically sorted on that key. Because the data

file is sorted, the index needs only one entry per block (sparse index), which keeps the index small and fast to

search.

A secondary index is built on a field that is not used to physically order the file. Since the underlying data is not

sorted by this field, the index must generally be dense — containing one entry for every record — so it can point

directly to each matching record.

Example(s)

-

roll_no in a Students table sorted by roll_no uses a primary (sparse) index.

-

An index on email or phone_number in the same table, where rows are not physically sorted by those

fields, must be a secondary (dense) index.

Justification / Critical Analysis

Primary indexes offer better storage efficiency and search speed but are restricted to the sorting attribute of the

file, of which there can only be one. Secondary indexes trade extra storage for flexibility, letting applications

efficiently query on any commonly searched attribute (e.g., searching by email even though the file is sorted by

roll number).

Q4. Static Hashing vs Dynamic Hashing

Bloom's Level: 4 – Analyze

Comparison Table

Aspect

Static Hashing

Dynamic Hashing

Number of buckets

Fixed at file creation and does not
change

Grows or shrinks automatically as data
volume changes

Hash function

Fixed function producing a fixed range of
bucket addresses

Extensible function (often uses extra bits
of the hash value) that adapts to bucket
growth

Handling growth

Bucket overflow chains form when data
exceeds capacity, degrading
performance

New buckets are added on demand,
avoiding long overflow chains

Reorganization cost

Full file reorganization needed if resized
(expensive)

Only local restructuring (e.g., splitting
one bucket) needed

Performance over time

Degrades as overflow chains grow

Stays roughly constant since bucket

count adapts to load

Complexity

Simple to implement

More complex — needs directory

structures (as in extendible hashing)

Explanation

Static hashing uses a hash function that maps a fixed number of buckets, decided once when the file is created.

If the number of records grows beyond the original capacity, records collide and are placed in overflow chains,

which gradually slows down retrieval.

Dynamic hashing addresses this limitation by allowing the number of buckets to grow or shrink as the data

volume changes, typically by using an expandable directory and progressively longer hash-value prefixes to

route records to buckets.

Example(s)

-

A static-hash table sized for 1,000 employee records will start performing poorly with long overflow

chains once the company grows to 5,000 employees.

-

A dynamic (extendible) hash index automatically splits the overloaded bucket and doubles the directory

as needed, keeping lookups close to $O(1)$ even as the dataset grows.

Justification / Critical Analysis

Static hashing is simpler and has lower overhead, making it suitable for applications with a known, stable record

count. Dynamic hashing is essential for databases with unpredictable or rapidly growing data, since it maintains

near-constant-time access without requiring a full file reorganization.

Q5. B-Tree Indexing vs B+ Tree Indexing

Bloom's Level: 4 – Analyze

Comparison Table

Aspect

B-Tree Indexing

B+ Tree Indexing

Data storage location

Both keys and data (or record pointers)

are stored in internal AND leaf nodes

Data pointers exist only in leaf nodes;

internal nodes store only keys for

routing

Leaf node linking

Leaf nodes are not linked to each other

Leaf nodes are linked in a sequence

(linked list), enabling efficient range

scans

Search for a single record

Can be faster since data may be found

at an internal node before reaching a

leaf

Always traverses to a leaf node, giving

consistent, predictable search time

Range queries

Less efficient — no direct link between

leaves, requires re-traversal

Very efficient — simply follow leaf-

level linked list

Storage efficiency

Lower fan-out since internal nodes also

carry data

Higher fan-out since internal nodes

carry only keys, so trees are shallower

Common usage

Less common in modern RDBMS

Widely used — default index structure

in MySQL (InnoDB), PostgreSQL, Oracle

Explanation

A B-Tree stores both keys and associated data pointers at every level of the tree (internal and leaf), which can

make single-record lookups slightly faster if the target happens to reside near the root, but it complicates

sequential/range access since data is scattered across levels.

A B+ Tree restricts actual data pointers to leaf nodes only; internal nodes act purely as a routing index. Critically,

all leaf nodes are linked together, so once a range query locates its starting point, it can scan sequentially

forward without re-traversing the tree — a major advantage for large-scale database applications.

Example(s)

-

MySQL's InnoDB storage engine and PostgreSQL both implement B+ Trees for primary and secondary

indexes precisely because most database queries (BETWEEN, ORDER BY, >, <) benefit from fast range

scans.

Justification / Critical Analysis

For large-scale applications with frequent range queries, sorting, and reporting, B+ Trees are almost always

preferred because of their linked-leaf structure and greater fan-out (shallower tree, fewer disk I/Os). B-Trees

may still be acceptable when workloads are dominated by single-key exact lookups and range queries are rare,

but in practice virtually all major relational database systems adopt B+ Trees as the default index structure.

Q6. Linear Hashing vs Extendible Hashing

Bloom's Level: 4 – Analyze

Comparison Table

Aspect

Linear Hashing

Extendible Hashing

Directory structure

No separate directory — buckets are

split in a predetermined linear order

Uses a directory of pointers to buckets,

indexed by bit-prefixes of the hash

Aspect

Linear Hashing

Extendible Hashing

Bucket splitting order

Buckets split in round-robin sequence

regardless of which bucket overflowed

Only the specific overflowing bucket is split

Overflow handling

Uses overflow chaining until its turn to split arrives

Directory doubling (if needed)

immediately accommodates the split

Directory growth

Not applicable — no directory to grow

Directory may double in size when local depth exceeds global depth

Memory overhead

Lower — no directory to maintain

Higher — must store and potentially resize the directory

Predictability of splits

Predictable, systematic split order

Split location is data-driven (based on which bucket is full)

Explanation

Linear hashing avoids the overhead of maintaining a directory by splitting buckets in a fixed, predetermined

round-robin order — one bucket at a time — regardless of which bucket actually overflowed, relying on

temporary overflow chains in the interim.

Extendible hashing maintains an explicit directory of pointers to buckets, addressed using the leading bits of a

record's hash value. When a specific bucket overflows, only that bucket is split; if its local depth exceeds the

directory's global depth, the directory itself doubles in size to accommodate finer addressing.

Example(s)

-

Linear hashing is well suited to file systems that want predictable, low-overhead growth without an

auxiliary directory.

-

Extendible hashing is used in database index structures where quick, targeted splitting of only the

overloaded bucket is valuable, at the cost of directory memory.

Justification / Critical Analysis

Both techniques aim to support dynamic growth without full file reorganization, but they trade off differently:

linear hashing keeps memory overhead low with a simple, systematic splitting scheme but may temporarily

tolerate overflow chains; extendible hashing reacts immediately and precisely to the bucket that actually

overflowed but requires extra memory for the directory, which can double in size during heavy growth.

Q7. Dense Indexing vs Sparse Indexing

Bloom's Level: 4 – Analyze

Comparison Table

Aspect

Dense Indexing

Sparse Indexing

Index entries

One index entry for every record in the
data file

One index entry per data block (not per
record)

Memory usage

Higher — index size grows
proportionally with the number of
records

Lower — index size grows only with the
number of blocks

Aspect

Dense Indexing

Sparse Indexing

Access speed (point query)

Faster — index directly points to the exact record

Slightly slower — index points to a block, then a scan within the block is needed

Applicability

Can be built on sorted or unsorted files

Requires the data file to be sorted on the indexed field

Storage trade-off

Uses more disk space for the index structure

More space-efficient, especially for large files

Explanation

A dense index contains an index record for every single search-key value in the data file, so it can directly locate

any record without scanning, but this means the index grows as large as the number of records.

A sparse index contains an entry only for some records — typically one per disk block — so it is much smaller

and can often be held in memory, but locating a specific record additionally requires a linear scan within the

located block.

Example(s)

-

A dense index on a Customers table's customer_id allows O(1)-style direct lookup of any customer, at

the cost of index size equal to the table size.

-

A sparse index on a sorted Orders table (one entry per block) saves substantial space when the table has

millions of rows spread over relatively few blocks.

Justification / Critical Analysis

The decision is essentially a space-versus-directness trade-off. Dense indexing favors raw lookup speed at the

cost of memory usage, making it attractive for smaller tables or when the file is unsorted. Sparse indexing favors

compactness and is only viable on a file already sorted on the indexed attribute, making it the practical choice

for very large, sorted data files where index memory footprint matters.

Q8. Indexed Sequential File Organization vs Direct File Organization

Bloom's Level: 4 – Analyze

Comparison Table

Aspect

Indexed Sequential File Organization

Direct (Random) File Organization

Access method

Combination of sequential access

(data sorted by key) plus an index for

direct access

Records accessed directly via a
computed address (usually a hash
function on the key)

Data ordering

Physically sorted on the key field

No particular physical order required

Retrieval

Supports both sequential (range) scans
and fast direct lookup via index

Very fast direct/point lookup, but poor
for sequential/range access

Insertion/Deletion

Moderate cost — may need overflow
areas and periodic reorganization to
keep the index and order consistent

Generally fast, since address is
computed directly, but collisions may
need handling

Aspect

Indexed Sequential File Organization

Direct (Random) File Organization

Storage overhead

Extra space needed for the index

structure

Minimal extra structure, though hash

collisions may waste space

Typical use case

Banking, payroll — need both batch

(sequential) processing and individual

account lookup

Real-time transaction systems needing

fast single-record access, e.g., session

lookup

Explanation

Indexed sequential file organization keeps the data file physically sorted by key (like sequential organization) but

adds an index on top, giving the best of both worlds: efficient sequential/batch processing and reasonably fast

direct access via the index, at the cost of extra index storage and reorganization overhead when the file changes

frequently.

Direct (random / hashed) file organization computes a record's storage address directly from its key using a hash

function, giving very fast point lookups without needing an index, but it sacrifices any notion of physical

ordering, making range queries and sequential processing inefficient.

Example(s)

-

A bank's account master file is often indexed-sequential — sorted by account number for month-end

batch reports, but indexed for a teller to instantly pull up one account.

-

A session-store keyed by a randomly generated session ID benefits from direct/hashed organization

since sessions are always looked up individually, never scanned as a range.

Justification / Critical Analysis

Indexed sequential organization is preferable when an application needs a mix of range/batch processing and

individual lookups, whereas direct file organization is preferable when the workload is dominated by single-key

point access and ordering/range queries are unimportant. The limitation of indexed-sequential is the overhead

of maintaining both the sort order and the index during frequent updates; the limitation of direct organization is

its complete unsuitability for range-based or sorted reporting.

Q9. Single-Level Indexing vs Multi-Level Indexing

Bloom's Level: 4 – Analyze

Comparison Table

Aspect

Single-Level Indexing

Multi-Level Indexing

Structure

One flat index mapping key values
directly to data block/record
addresses

A hierarchy of indexes — an index
built on top of another index (e.g.,
outer index, inner index)

Search cost

Search time grows with index size
($O(\log n)$ with binary search, but the
whole index may not fit in memory)

Search cost reduced further since only
the small top-level index needs
scanning to locate the relevant portion
of the lower index

Suitability for large files

Becomes inefficient as the file — and
hence the index — grows very large

Scales well for very large files since
higher levels stay small enough to fit in
memory

Aspect

Single-Level Indexing

Multi-Level Indexing

Structure basis

Simple linear or binary-searchable list

Typically implemented as a tree

structure (e.g., B+ Tree), often multiple

levels deep

Maintenance complexity

Simpler to build and maintain

More complex — insertions/deletions

may cascade changes across levels

Explanation

A single-level index is one flat structure directly mapping key values to record/block addresses.

While a binary

search on this index is efficient in theory, once the file becomes very large, even the index itself may become too

large to fit in memory, slowing down every access.

Multi-level indexing addresses this by building an index on the index — a smaller, higher-level (outer) index that

helps quickly locate the relevant section of the lower-level (inner) index. This hierarchy can continue for as many

levels as needed, and is the principle underlying B-Tree/B+ Tree indexes used in modern databases.

Example(s)

-

A single-level index on a modest 10,000-row table performs well since the index itself is small.

-

A multi-level (B+ Tree) index on a table with 100 million rows keeps the tree only 3–4 levels deep, so any

record can be found in just 3–4 disk I/Os instead of a large binary search over a huge flat index.

Justification / Critical Analysis

As data volume grows, a single-level index eventually loses its efficiency advantage because the index itself

becomes too large for fast memory-resident search. Multi-level indexing directly solves this scalability problem

by keeping the top levels of the hierarchy small and memory-resident, which is precisely why virtually all

production-grade relational databases use multi-level tree-based indexes (B+ Trees) rather than flat single-level

indexes for large tables.

Q10. Hashing Techniques vs Indexing Techniques for Exact-Match and Range Queries

Bloom's Level: 5 – Evaluate

Comparison Table

Aspect

Hashing Techniques

Indexing Techniques (B-Tree / B+ Tree)
--

Exact-match queries

Excellent — average $O(1)$ lookup by computing the hash directly
--

Good — $O(\log n)$, slower than hashing but still efficient
--

Range queries (e.g., BETWEEN, >, <)

Poor — hash values do not preserve order, so ranges require scanning many/all buckets

Excellent — sorted tree/leaf structure allows efficient sequential range scans

Sorting / ORDER BY support

Not supported directly — no inherent ordering

Naturally supported since data/keys are kept sorted

Worst-case performance

Degrades under heavy collisions/overflow chains to $O(n)$

Bounded — $O(\log n)$ even in the worst case for balanced trees

Aspect

Hashing Techniques

Indexing Techniques (B-Tree / B+ Tree)

Best-fit scenario

Point lookups on high-cardinality

unique keys (e.g., primary key equality search)

Range-heavy, analytical, and reporting workloads

Explanation

For exact-match queries, hashing techniques are generally superior because a well-designed hash function

computes the target bucket directly in constant average time, avoiding the multi-level traversal that a tree index

requires.

For range queries, indexing techniques (particularly B+ Trees) clearly outperform hashing, because hashing

intentionally scrambles key order to distribute values evenly across buckets — this is exactly what makes it fast

for equality lookups but useless for retrieving all values between two bounds, since matching records could be

scattered across every bucket.

Example(s)

-

Evidence from the SQLite demonstration (see Q2) confirms this: an indexed range query on roll_no

(SEARCH ... rowid>? AND rowid<?) used the ordered B-Tree-like primary key structure efficiently, something a pure hash index cannot do without a full bucket scan.

-

A database's WHERE customer_id = 12345 (equality) is ideal for a hash index, whereas WHERE order_date BETWEEN '2026-01-01' AND '2026-06-30' (range) requires a B+ Tree index to be efficient.

Justification / Critical Analysis

Evaluating both techniques together: neither is universally superior — the correct choice depends entirely on

the query workload. Hashing should be chosen when the dominant access pattern is exact-match lookups on

high-cardinality keys, since it offers near-constant-time performance unmatched by tree indexes. Indexing (B-

Tree/B+ Tree) should be chosen whenever range queries, sorting, or partial-match queries are common, since

only an ordered structure supports these efficiently. Many production databases pragmatically use both: a B+

Tree as the primary/clustered structure for ordering and range support, supplemented with hash indexes on

specific high-traffic equality-lookup columns.